

Distributed Computing 1

Distributed Systems, Dask Architecture & Local Clusters

Week 6 – 19 March 2026

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University





Distributed Computing 1

Today's Agenda

- ▶ **Theory** (30 min): Distributed computing, Dask architecture, communication cost model
- ▶ **Studio** (155 min):
 - ▶ E1: Dask delayed — lazy evaluation & Monte Carlo π
 - ▶ E2: LocalCluster & dashboard
 - ▶ E3 (*optional*): Dependent task graphs — two-pass normalisation
 - ▶ MP2 M1: Dask Mandelbrot (local)
 - ▶ MP2 M2: Chunk size sweep
 - ▶ MP2 M3: Full benchmark (all methods)
- ▶ **Soft wrap-up** (~15:00): leave if finished or keep working until 16:15

Pre-check: `python -c "import dask; print(dask.__version__)"` — should print a version number

MP2 deadline: Mon 7 Apr 23:59

Agenda



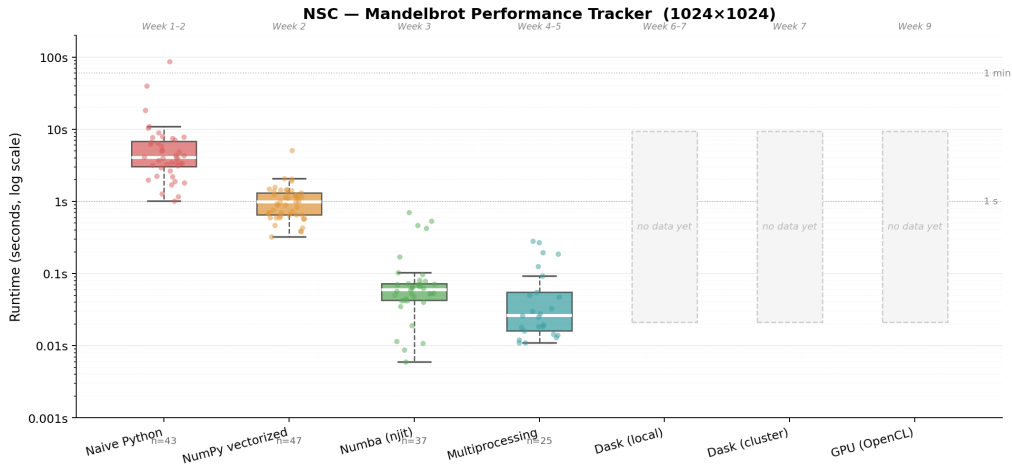
Distributed Computing Concepts

Studio

Wrap-up

Performance Tracker

Week 6 — 164 submissions



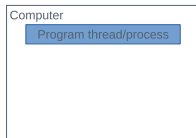


Distributed Computing Concepts

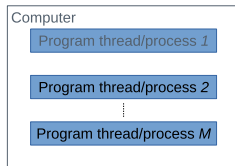
Distributed Computing

Parallel vs. Distributed

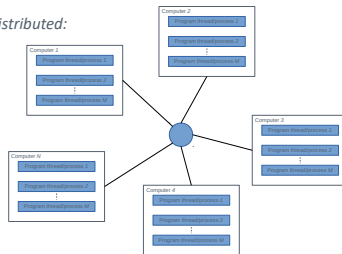
Sequential:



Parallel:



Distributed:



Aspect	Parallel	Distributed
Hardware	Multiple cores, one machine	Multiple machines, networked
Memory	Shared — all cores see same RAM	Separate — each node has own RAM
Communication	Memory bus (ns– μ s)	Network (10s of μ s – ms)
Scale	2–64 cores typically	Hundreds to thousands of cores

When do you need distributed?

- ▶ Data does not fit in one machine's memory
- ▶ Need more compute than one machine provides



Distributed Computing

What Changes at Scale

Moving from shared memory to distributed introduces new challenges:

- ▶ **No shared memory:** data must be explicitly sent between nodes; every worker holds only the slice it needs
- ▶ **Network is the bottleneck:** Ethernet latency ($\sim 50 \mu s$) is $\sim 500\times$ higher than RAM; tasks must be large enough to amortise communication cost (the $\alpha + \beta n$ model from Week 2 now dominates)
- ▶ **Coordination overhead:** a scheduler must track task dependencies, assign work, and collect results — adds latency even on a single machine
- ▶ **Partial failures:** individual nodes or links can fail independently; fault-tolerant frameworks detect and re-schedule failed tasks automatically
- ▶ **Data locality:** moving data to the computation is often cheaper than the reverse; good schedulers run tasks where data already resides

These challenges make distributed programming harder than shared-memory parallelism — but they are manageable when workloads are large enough to justify the overhead.

Distributed Computing

The TOP500 List

The TOP500 (top500.org) ranks supercomputers by HPL benchmark (dense LU factorisation). Updated twice yearly since 1993.

November 2024 — top entries:

Machine	EFlop/s	Nodes	Total GPUs	Accelerator
El Capitan	1.742	11,040	44,160	AMD Instinct MI300A
Frontier	1.206	9,408	37,632	AMD Instinct MI250X
Aurora	1.012	10,624	63,744	Intel Ponte Vecchio

Key observations:

- ▶ Three **exascale** machines ($> 1 \text{ EFlop/s} = 10^{18} \text{ Flop/s}$); first was Frontier (2022)
- ▶ **Highly distributed**: each machine is $\sim 9,000$ – $11,000$ compute nodes, each node carrying 4–6 GPU accelerators — (40,000–60,000) GPUs in total; no single chip comes close alone
- ▶ All use **high-speed interconnect** (InfiniBand or HPE Slingshot at 200 Gb/s) — the hardware that makes the $\alpha + \beta n$ communication cost model relevant (covered later today)
- ▶ Performance has grown $\sim 1,000\times$ per decade; the #1 machine in 1993 achieved $\sim 60 \text{ GFlop/s}$

HPL measures peak performance on a compute-bound workload; real applications typically achieve 10–60% of this figure. See Eijkhout 2.11.4 for background on the benchmark.

Distributed Computing

Capability vs. Capacity Computing

Capability computing

- ▶ One large job uses the whole machine
- ▶ Metric: time-to-solution
- ▶ Example: a single CFD or MD simulation too large for fewer nodes
- ▶ Tightly coupled — workers communicate constantly (MPI)

Capacity computing

- ▶ Many independent jobs share the machine
- ▶ Metric: throughput — jobs completed per unit time
- ▶ Examples: parameter sweeps, Monte Carlo ensembles, ML cross-validation
- ▶ Embarrassingly parallel — no inter-job communication (Hadoop, Spark, Dask, task queues)

Most research computing in EE/CS/Data is capacity computing.

Eijkhout 2.11.2. MapReduce (Hadoop, Spark, Dask) formalises the capacity computing model — `map` distributes work, `reduce/gather` collects results.

HPC Infrastructure

Job Schedulers: SLURM

Clusters are shared resources: a **job scheduler** queues resource requests, allocates nodes when available, and enforces fair-share priorities. **SLURM** (Simple Linux Utility for Resource Management) is the standard on academic and national HPC systems, including **AAU AI Lab** and DeiC facilities. SLURM handles both **batch jobs** (`sbatch` — fire and forget) and **interactive allocations** (`salloc` — run Jupyter or Dask on reserved nodes).

Minimal job script (`job.sh`):

```
#!/bin/bash
#SBATCH --job-name=mandelbrot
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --time=01:00:00
#SBATCH --mem=8G

conda activate nsc
python mandelbrot.py
```

Key commands:

- ▶ `sbatch job.sh` — submit job to queue
- ▶ `squeue -u $USER` — list your jobs
- ▶ `scancel <jobid>` — cancel a job
- ▶ `sinfo` — show partition/node status

`#SBATCH` directives declare the resources needed (cores, memory, walltime, GPU count) before the cluster allocates them.

In L7 we use **Strato** (OpenStack VM-based, no scheduler — you manage nodes directly). SLURM is what you will encounter on AI Lab, national facilities, and most industry HPC clusters.

HPC Infrastructure

Batch vs. Interactive

Batch (`sbatch`)

- ▶ Job script submitted to queue; runs unattended
- ▶ Output written to file (`slurm-JOBID.out`)
- ▶ Notified when done (`--mail-type=END`)
- ▶ Best for: long runs, parameter sweeps, overnight jobs

Interactive (`salloc`)

- ▶ Waits in queue; grants a shell on allocated nodes
- ▶ Dedicated *interactive partition*: stricter walltime limit, higher base priority → fast turnaround
- ▶ Best for: development, debugging, Jupyter analysis
- ▶ Notified when allocation starts (`--mail-type=BEGIN`)

SLURM scheduling policy — why modest requests start faster:

- ▶ **Backfill scheduling:** SLURM fills idle gaps between reserved slots with jobs that fit the available window. A 1-node, 2-hour job fits many gaps; a 100-node, 48-hour job must wait for a contiguous slot.
- ▶ **Fair-share:** priority is weighted by your group's recent usage relative to its allocation — heavy users are deprioritised until the cluster rebalances.
- ▶ **Practical rule:** request only what you need. Over-requesting nodes or walltime delays your own job and reduces fair-share credit for future jobs.

Distributed Frameworks

MPI vs. Dask

MPI has been the HPC standard since 1994 and remains dominant in scientific computing. Dask is newer and trades some overhead for a much simpler programming model.

Aspect	MPI	Dask
Model	Explicit message passing (send/recv)	Lazy task graph; automatic scheduling
Python API	<code>mpi4py</code>	<code>dask.distributed</code>
Task overhead	Near hardware limits (μ s)	$\sim 1\text{--}50$ ms per task
Interface	Low-level; programmer controls all messages	High-level; NumPy/pandas-like
Scaling	Millions of cores	Thousands of nodes
Fault tolerance	None	Built-in
Best for	Tightly coupled numerics (CFD, FEM, MD)	Data analytics, ML, irregular task graphs

→ When to use each tool: next slide.

Distributed Frameworks

When to Use Which Tool

	Single machine	Cluster
Independent Tightly coupled	Numba parallel=True threading	MPI CFD · FEM · MD
	NumPy · Numba Multiprocessing <i>L01–L05</i>	Dask <i>L06: LocalCluster</i> <i>L07: Strato cluster</i>

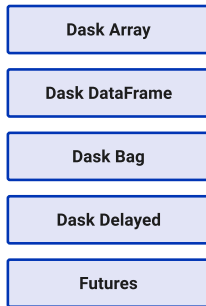
Mandelbrot is embarrassingly parallel — no inter-task communication; scale is the only axis. **MP2:** LocalCluster today (L06); same code on Strato cluster in L07.

Dask

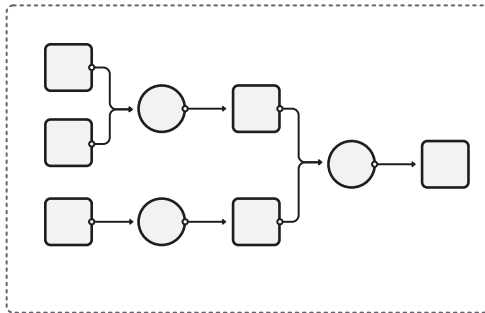
User Interfaces

Collections

(create task graphs)



Task Graph



Schedulers

(execute task graphs)

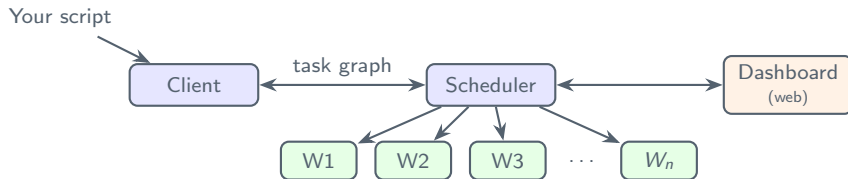
Single-machine
(threads, processes,
synchronous)

Distributed

This course uses **Delayed** (MP2 computations) and **Futures** (dashboard/live monitoring).
Array/DataFrame/Bag are high-level collections for data-heavy workloads.

Distributed Computing

Dask Architecture



Components:

- ▶ **Client:** Your Python code
- ▶ **Scheduler:** Manages task graph, assigns work
- ▶ **Workers:** Execute tasks
- ▶ **Dashboard:** Real-time monitoring (web)

Two modes:

- ▶ **Local (today):** All components on your laptop
— LocalCluster
- ▶ **Cluster (L7):** Scheduler + workers on Strato
— same code, more resources

Dask Core Concepts

Lazy Evaluation and `dask.delayed`

A normal function call executes *immediately* and returns a result.

`dask.delayed` wraps a function so that calling it returns a **Delayed object** instead — the computation is *recorded*, not run.

- ▶ **Build phase:** call delayed functions to construct the task graph — no workers are used yet
- ▶ **Execute phase:** `dask.compute(*tasks)` hands the graph to the scheduler, which runs all tasks and returns results
- ▶ **Inspect:** `dask.visualize(*tasks)` renders the graph as a PNG *without* executing — use this to verify structure before running

Analogy: a to-do list — you write down all tasks first, hand the list to the scheduler, then it executes them.

```
from dask import delayed
import dask

def add(a, b):
    return a + b

# Normal call -- executes now
result = add(1, 2)          # result = 3

# Delayed call -- records intent
lazy = delayed(add)(1, 2)   # Delayed object
print(lazy)                 #
                             Delayed('add-...')

# Nothing has run yet -- build more tasks
tasks = [delayed(add)(i, i) for i in range(8)]

# Execute all at once
results = dask.compute(*tasks)
# results = (0, 2, 4, 6, 8, 10, 12, 14)

# Visualise without running
dask.visualize(*tasks, filename='graph.png')
```


Dask Core Concepts

Task Graphs and Futures

Task graph (DAG — Directed Acyclic Graph):

- ▶ Nodes = individual function calls; edges = data dependencies
- ▶ Built automatically from delayed calls — no manual wiring
- ▶ Scheduler reads the graph: runs independent tasks in parallel, respects dependencies (as in E3 today)
- ▶ `dask.visualize()` renders it — useful for debugging and for understanding what Dask is actually doing

Two execution interfaces:

	<code>compute()</code>	<code>submit()/map()</code>
Returns:	tuple of results	<code>Future(s)</code>
Blocking call?	yes, until done	no — fire & forget
Use when:	need all results	overlap work / partial results

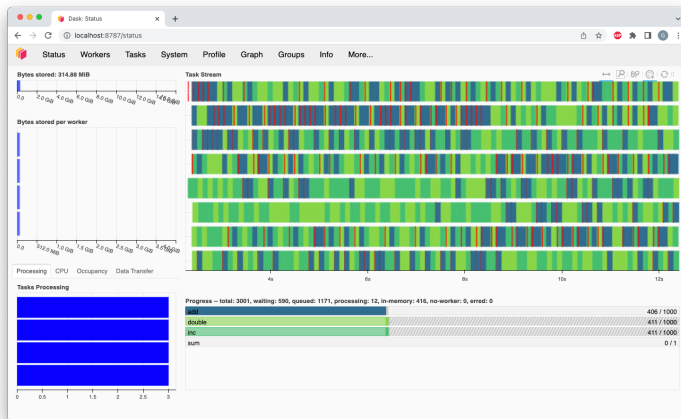
API quick reference:

- ▶ `delayed(f)(x)` records the call; no execution yet
- ▶ `dask.compute(*tasks)` submits graph, blocks until done
- ▶ `client.submit(f, x) → Future`; returns immediately
- ▶ `client.map(f, items)` dispatches many tasks
- ▶ `future.result()` blocks when you need the value

For MP2: use `dask.compute()` for timing — it blocks until all chunks finish (submit() would return before work is done). The dashboard shows task execution for both interfaces.

Dask Dashboard

Real-time task monitoring



Communication Cost Model

$\alpha + \beta n$ — Eijkhout 1.3.2

Recall from Week 2: Data transfer time is modelled as:

$$T(n) = \alpha + \beta \cdot n$$

- ▶ α = **latency** (startup cost per message, independent of size)
- ▶ β = **inverse bandwidth** (time per byte)
- ▶ n = message size (bytes)

Week 2: memory transfers (nanoseconds).

Week 6: network communication — same model, very different scale:

	Latency α	Bandwidth
L1 Cache (per core)	~ 1 ns	150-400 GB/s
RAM	~ 100 ns	30-100 GB/s
Network (Ethernet)	~ 50 μ s	1-10 GB/s
Network (Infiniband)	~ 1 μ s	25-200 GB/s

Ethernet latency is $\sim 500\times$ higher than RAM latency.

Implication: tasks must be large enough to amortise α .

Communication Cost Model

Chunk Size and Scheduling Overhead

General model (k chunks, n_w workers; pipelining partially overlaps terms):

$$T_{\text{wall}} \approx \underbrace{\frac{T_{\text{serial}}}{n_w}}_{\text{compute}} + k \cdot \left(\underbrace{\alpha_{\text{eff}}}_{\text{latency}} + \underbrace{\beta \cdot s}_{\text{data transfer}} \right)$$

Two simplifications apply to Mandelbrot:

- **Compute-bound** ($\sim 4 \text{ B/px output}$): βs negligible — data transfer term drops out
- **Dask pipelining**: α_{eff} hidden for large tasks — latency term drops out (only visible in overhead regime where task time $\approx \alpha_{\text{eff}}$)

$\Rightarrow$ For large chunks: $T_{\text{wall}} \approx T_{\text{serial}}/n_w$ (pure parallel speedup).

	Dask local (IPC)	Dask cluster (Ethernet)
α_{eff}	$\sim 1\text{--}5 \text{ ms}$	$\sim 10\text{--}50 \text{ ms}$
Task time needed	$\gg 5 \text{ ms}$	$\gg 50 \text{ ms}$

Dask adds Python serialisation and scheduler round-trips on top of raw hardware α . Higher α_{eff} on a cluster pushes the sweet spot towards *fewer, larger* chunks.

Chunk Size: Three-Way Trade-Off

Three constraints pull in *opposite* directions — the sweet spot is hardware-specific.

Constraint	Generic rule	M1 (33 B/px)	Max	example
Cache fit (L02)	Working set $\lesssim$ L2/worker	$\lesssim$ 300 rows/chunk		
Task overhead (L04)	Task time $\gg \alpha_{\text{eff}}$	$\gtrsim$ 64 rows/chunk		
Worker alignment (L04)	n_{chunks} divisible by n_w	e.g. multiples of 8 for 8 workers		

Cache recap (L02):

- ▶ L1: 64–192 KB/core, private (performance vs. efficiency cores)
- ▶ L2: private on x86; per-cluster on Apple Silicon (varies by chip)
- ▶ LLC/SLC: tens of MB, shared

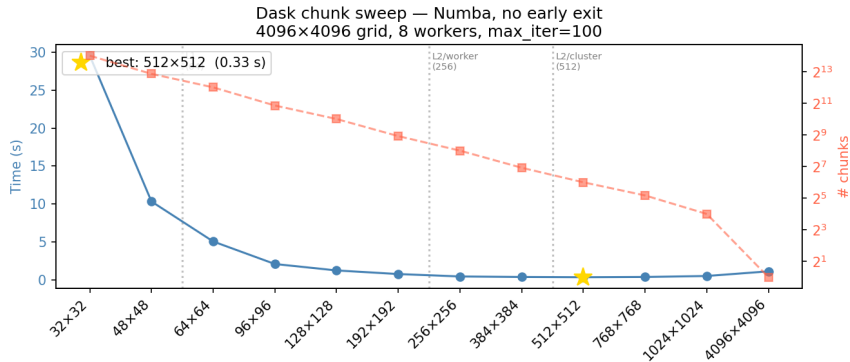
Straggler effect (L04):

- ▶ Last scheduling round has $r < n_w$ active workers
- ▶ Effect largest when k (total chunks) is small
- ▶ Set n_{chunks} to a multiple of n_w to avoid a partial final round

⇒ Measure empirically on your hardware — next slide shows M1 Max results.

Chunk Size in Practice

4096×4096 Mandelbrot, Numba, 8 workers, M1 Max



- **Overhead regime (32–256):** too many chunks; scheduling cost $\gg$ compute per chunk
- **Sweet spot (512):** 64 chunks = 8 full rounds, no straggler; working set fits in L2
- **Straggler penalty (768):** 36 chunks not divisible by 8 — slower than both 512 and 1024
- **Worker starvation (≥ 1024):** too few chunks; single chunk (4096) uses one core only

Early Exit: Why Chunk Shape Matters

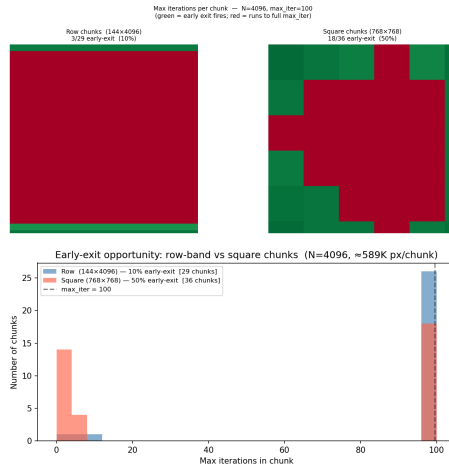
Strategy: after each iteration step, check if *all* pixels in the chunk have diverged — if so, break the loop early — worker becomes free to work on next chunk.

Only fires when every pixel in the chunk has escaped.
 $\Rightarrow$ chunks entirely outside the set finish in a few iterations; chunks overlapping the interior run to `max_iter`.

Why shape matters:

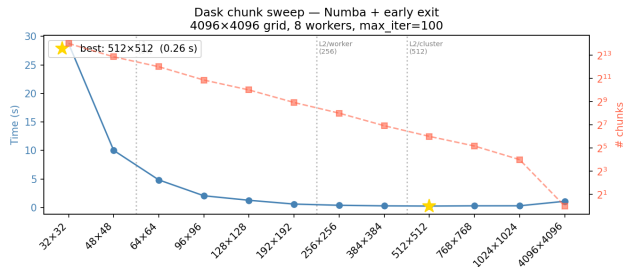
- **Row band** ($h \times N$): spans the full x-range $\rightarrow$ always contains interior pixels $\rightarrow$ early exit $\approx 10\%$
- **Square** ($cs \times cs$): localised in both axes $\rightarrow$ corner tiles lie in exterior $\rightarrow$ early exit $\approx 50\%$

Same pixel budget; shape is the only difference.



Chunk Size with Early Exit

4096×4096 Mandelbrot, 8 workers, M1 Max



- **Overhead regime (32–256):** scheduling cost dominates; early exit gives negligible benefit here
- **Sweet spot (512):** same as without early exit — early exit gives $\sim 1.3\times$ speedup (exterior chunks escape early)
- **Straggler penalty (768):** early exit narrows the gap but 768 still slower than 512
- **Single chunk (4096):** all work on one core; no parallelism
- **Benchmarking:** always on AC power (battery throttles CPU); close the dashboard tab during timing (adds scheduler load)



Studio

Studio Overview

Exercises (no hand-in):

- E1** Dask delayed — lazy evaluation & Monte Carlo π
- E2** LocalCluster & dashboard — watch tasks execute in real time
- E3** (*optional*) Dependent task graphs — two-pass normalisation pipeline

MP2 Milestones:

- M1** Dask Mandelbrot (local): implement with `dask.delayed` + Numba; time it
- M2** Chunk size sweep: vary `n_chunks`, find minimum wall time and LIF
- M3** Full benchmark: all implementations side-by-side; submit to Performance Tracker

Exercise 1

Dask Delayed — Lazy Evaluation

Warm up with `dask.delayed`: understand what lazy evaluation means in practice.

1. Run the serial Monte Carlo π (see code example) and note the time.
2. Wrap `monte_carlo_chunk` with `delayed`; call it `n_chunks` times.
Observe: the function does *not* execute yet — no output.
3. Visualise the task graph: `dask.visualize(*tasks, filename='graph.png')`
Open the PNG — which tasks can run in parallel?
4. Call `dask.compute(*tasks)` and time it. Compare to serial.
5. Vary `n_chunks` (4, 8, 16, 32): how does overhead scale with very small chunks?

Note: without a `LocalCluster`, `dask.compute()` uses a single-threaded scheduler — Dask may be *slower* than serial here; that is expected. The point of E1 is lazy evaluation and task graph structure, not speedup. E2 adds a `LocalCluster`.

✓ **Done?** What speedup (or slowdown) did you observe, and why? → discuss with a neighbour

Exercise 1

Code example

```
import dask, random, time, statistics
from dask import delayed

def monte_carlo_chunk(n_samples):
    inside = 0
    for _ in range(n_samples):
        x, y = random.random(), random.random()
        if x*x + y*y <= 1:
            inside += 1
    return inside

total, n_chunks = 1_000_000, 8
samples = total // n_chunks

# Serial baseline
t0 = time.perf_counter()
results = [monte_carlo_chunk(samples) for _ in range(n_chunks)]
t_serial = time.perf_counter() - t0
print(f"Serial: {t_serial:.3f} s   pi={4*sum(results)/total:.4f}")

# Dask delayed -- task graph is built, not executed yet
tasks = [delayed(monte_carlo_chunk)(samples) for _ in range(n_chunks)]
t0 = time.perf_counter()
results = dask.compute(*tasks)
t_dask = time.perf_counter() - t0
print(f"Dask:   {t_dask:.3f} s   pi={4*sum(results)/total:.4f}")

# Visualise (requires: conda install python-graphviz)
dask.visualize(*tasks, filename='task_graph.png')
```

Exercise 2

LocalCluster & Dashboard

Use a real Dask scheduler and watch task execution live.

1. Create a `LocalCluster` and connect a `Client` (see code example).
2. Open the dashboard URL in your browser (printed on startup).
3. Rerun E1 Monte Carlo π while watching the **Task Stream** panel.
4. Try varying `n_workers` (2, 4, 8) — does the task stream change?
5. Observe: are all workers busy? Are there any stragglers?

Dashboard panels to explore:

- ▶ *Task Stream*: real-time execution per worker
- ▶ *Workers*: CPU and memory per process
- ▶ *Progress*: completion percentage of current compute

✓ **Done?** What did you see in the task stream? → discuss with a neighbour

Exercise 2

Code example

```
from dask.distributed import Client, LocalCluster
import dask
from dask import delayed

# Create local cluster; start with max workers -- scale() adjusts without restarting
cluster = LocalCluster(n_workers=8, threads_per_worker=1)
client = Client(cluster)

print(f"Dashboard: {client.dashboard_link}")
# --> open the printed URL in your browser

# Rerun E1 tasks; LocalCluster scheduler takes over
tasks = [delayed(monte_carlo_chunk)(samples) for _ in range(n_chunks)]
results = dask.compute(*tasks)

# Vary n_workers: scale() resizes without restarting the scheduler
# (recreating LocalCluster while the browser is open breaks the dashboard)
cluster.scale(4); client.wait_for_workers(4) # now 4 workers
tasks = [delayed(monte_carlo_chunk)(samples) for _ in range(n_chunks)]
results = dask.compute(*tasks)

client.close(); cluster.close()
```

Exercise 3 (*optional*)

Dependent Task Graphs

So far every task has been independent. Real pipelines may have **cross-chunk dependencies** — Dask resolves them automatically.

Two-pass normalisation (classic fan-in / fan-out pattern):

1. **Stage 1 (parallel)**: generate each chunk; compute per-chunk maximum
2. **Stage 2 (fan-in)**: `global_max` — waits for *all* Stage 1 tasks
3. **Stage 3 (fan-out)**: normalise each chunk using `global_max` — parallel again

You write *zero* synchronisation code. Dask infers the execution order from which delayed objects appear as arguments.

What to observe:

- ▶ **Task graph** (`task_graph_pipeline.png`): does the hourglass shape match the three-stage structure?
- ▶ **Task Stream** — **phases**: identify the two parallel bands and the single-task bottleneck; match each colour to a function (`generate`, `chunk_max`, `global_max`, `normalise`)
- ▶ **Task Stream** — **red boxes** (`transfer-*`): unavoidable — data *must* cross worker boundaries at fan-in/fan-out points. This is the βs term. NumPy minimises s ; the transfers remain.
- ▶ **Timing**: how close to $T_\infty = T_{S1}/n_w + T_{S2} + T_{S3}/n_w$?

✓ **Done?** Identify the phases in the Task Stream → discuss with a neighbour

Exercise 3 (optional)

Code example

```
import time, numpy as np, dask
from dask import delayed
from dask.distributed import Client, LocalCluster

# Each function sleeps so stages are visible in the dashboard Task Stream
@delayed
def generate(seed, n):    time.sleep(0.3); return np.random.default_rng(seed).standard_normal(n)
@delayed
def chunk_max(data):     time.sleep(0.2); return float(np.max(np.abs(data)))
@delayed
def global_max(maxima):  time.sleep(0.2); return max(maxima)    # fan-in: waits for ALL chunk_max
@delayed
def normalise(data, g):  time.sleep(0.3); return data / g      # fan-out: parallel once gmax ready

if __name__ == '__main__':
    cluster = LocalCluster(n_workers=4, threads_per_worker=1)
    client  = Client(cluster); print(client.dashboard_link)

    chunks = [generate(i, 50_000) for i in range(8)]    # Stage 1a
    maxima  = [chunk_max(c)        for c in chunks]     # Stage 1b
    gmax    = global_max(maxima)    # Stage 2: fan-in
    normed  = [normalise(c, gmax)   for c in chunks]     # Stage 3: fan-out

    dask.visualize(*normed, filename='task_graph_pipeline.png')
    t0 = time.perf_counter()
    results = dask.compute(*normed)
    print(f"Wall time: {time.perf_counter()-t0:.2f} s")
    client.close(); cluster.close()
```


MP2 M1

Dask Mandelbrot — Local

Approach: wrap your existing Numba `mandelbrot_chunk` with `dask.delayed` — same row-chunk structure as L04/L05, one task per row-band:

```
dask.compute(*[delayed(f)(args) for args in chunk_args]) ← replaces pool.map(f, chunk_args)
```

1. Implement `mandelbrot_dask()` using `dask.delayed` (see code example).
2. Verify: `np.array_equal(ref, result)` against your serial output.
3. Warm up Numba JIT in *all* workers with `client.run()` before timing.
4. Time 3 runs; record `statistics.median()`.

Common pitfall: Do NOT iterate Dask arrays inside the `max_iter` loop (e.g. `Z = da.where(...)`). Each of the 100 iterations becomes a separate task — $\sim 100\,000$ tasks for $N = 1024$ — and overhead dominates.

`dask.delayed` wraps the entire Numba function as *one* atomic task per chunk.

✓ **Done?** Record time in performance notebook (MP2) → commit

MP2 M1

Code example

```
from dask import delayed
from dask.distributed import Client, LocalCluster
import dask, numpy as np, time, statistics
# mandelbrot_chunk: your @njit(cache=True) function from L04/L05
def mandelbrot_dask(N, x_min, x_max, y_min, y_max,
                    max_iter=100, n_chunks=32):
    chunk_size = max(1, N // n_chunks)
    tasks, row = [], 0
    while row < N:
        row_end = min(row + chunk_size, N)
        tasks.append(delayed(mandelbrot_chunk)(
            row, row_end, N, x_min, x_max, y_min, y_max, max_iter))
        row = row_end
    parts = dask.compute(*tasks)
    return np.vstack(parts)
if __name__ == '__main__':
    N, max_iter = 1024, 100
    X_MIN, X_MAX, Y_MIN, Y_MAX = -2.5, 1.0, -1.25, 1.25
    cluster = LocalCluster(n_workers=8, threads_per_worker=1)
    client = Client(cluster)
    client.run(lambda: mandelbrot_chunk(0, 8, 8, X_MIN, X_MAX,
                                       Y_MIN, Y_MAX, 10))

    times = []
    for _ in range(3):
        t0 = time.perf_counter()
        result = mandelbrot_dask(N, X_MIN, X_MAX, Y_MIN, Y_MAX, max_iter)
        times.append(time.perf_counter() - t0)
    print(f"Dask local (n_chunks=32): {statistics.median(times):.3f} s")
    client.close(); cluster.close()
```

MP2 M2

Chunk Size Sweep

Find the optimal chunk count for Dask local on your machine.

Sweep `n_chunks` over a range guided by the three-way trade-off (see theory slides).

For each setting: time 3 runs (median), compute $LIF = p \cdot T_p / T_1 - 1$.

1. Adapt your L05 Dask implementation to loop over `n_chunks` values: keep `LocalCluster` open across all measurements (start-up cost paid once).
2. Warm up Numba JIT in *all* workers with `client.run()` before the sweep loop.
3. Record: wall time and LIF per `n_chunks`; plot wall time vs. `n_chunks` (log scale).
4. Note: Dask has higher α per task than multiprocessing — expect the optimal `n_chunks` to be *smaller* than in L05.

Print table columns:

`n_chunks` | time (s) | vs 1x | speedup | LIF

Record:

`n_chunks_optimal`, `t_min`, `LIF_min`

Save plot as `dask_chunk_sweep.png`

Optional: Add block-level early exit to your Numba kernel: after each iteration step, check if all pixels in the chunk have diverged — if so, break the loop. Chunks entirely outside the Mandelbrot set then skip remaining iterations. Compare timing with and without this optimisation.

✓ **Done?** Record `n_chunks_optimal` and `LIF_min` in performance notebook (MP2) → commit

MP2 M3

Full Benchmark — All Implementations

Collect all timings and compare every implementation built so far.

Add a Dask row to your performance notebook:

Implementation	Time (s)	Speedup vs. naive
Naive Python		1×
NumPy		
Numba (@njit)		
Numba + multiprocessing		
Dask local		
Dask cluster	<i>(to be added in L7)</i>	

Reflection questions (address in your written reflection):

- ▶ How does Dask local compare to multiprocessing at the same worker count?
- ▶ What does the overhead difference tell you about when to choose each tool?

Submit: Add your Dask local result to the Performance Tracker in Moodle.

✓ **Done?** Record all results in performance notebook (MP2) → commit



Wrap-up

Wrap-up

What We Covered Today

► Theory:

- Parallel vs. distributed computing; when to scale beyond one machine
- Dask architecture: client, scheduler, workers, dashboard
- Communication cost model: $\alpha + \beta n$ and its implication for chunk size

► Dask programming:

- Lazy evaluation with `dask.delayed`
- Task graph visualisation
- `LocalCluster` and the real-time dashboard

► MP2 progress:

- Multiprocessing ✓ (L4–5)
- Dask local ✓ (today)
- Dask cluster (L7 – Strato)

Before L7:

- Listen to podcast / read Eijkhout on cluster computing (see Moodle)
- Verify Strato access: `ssh <aau-id>@strato.claudia.aau.dk`
- Bring working Dask Mandelbrot code to L7

MP2 deadline: Mon 7 Apr 23:59